Universidade Federal do Maranhão
Centro de Ciências Exatas e Tecnologia — Departamento de Engenharia da Computação
Disciplina: Sistemas Distribuídos — Docente: Prof. Dr. Luiz Henrique Neves Rodrigues

Relatório Técnico de Evolução Arquitetural

Modernização com Middleware MOM, Virtualização Isolada em 6 VMs e Análise de Escalabilidade

Discentes: Eduardo dos Santos Oliveira, Fernando da Silva Costa, Herick Vinicius Pinheiro Da Conceição, Samara Santos Viegas e Vyktor Gabryel Costa de Souza

---

1 Introdução e Contexto das Alterações

O monitoramento contínuo de sinais vitais em Unidades de Terapia Intensiva (UTI) requer alta disponibilidade, processamento em tempo real e integridade absoluta dos dados médicos. O **Sistema Distribuído para Auxílio de Dados Hospitalares** foi concebido para coletar, enfileirar, balancear e persistir fluxos massivos de telemetria médica provenientes de sensores clínicos.

Na versão inicial do projeto, o sistema operava com comunicação síncrona direta via protocolo HTTP (Cliente → Balanceador → Servidores) e agrupava múltiplos servidores dentro de uma mesma Máquina Virtual compartilhada. Para mitigar riscos inerentes a essa abordagem, as alterações consolidadas concentraram-se em três eixos de evolução:

1. **Adoção de Middleware Orientado a Mensagens (MOM):** Incorporação do RabbitMQ para amortecimento de fluxo, desacoplamento temporal e garantia de entrega incondicional;
2. **Segregação Física em 6 Máquinas Virtuais (VMs):** Isolamento de cada componente em nós virtuais autônomos com endereçamento de rede independente, eliminando o compartilhamento de recursos;
3. **Atualização da Camada de Observabilidade Gráfica:** Ajuste da topologia visual para mapear em tempo real as 6 máquinas e os novos enlaces de mensageria.

2 Camada de Middleware Orientado a Mensagens (MOM)

2.1 Conceito de Middleware e o Papel do MOM

Em computação distribuída, o termo **Middleware** designa uma camada de software intermediária posicionada entre o sistema operacional e as aplicações finais. Sua função é viabilizar a comunicação e o gerenciamento de dados entre nós heterogêneos de maneira transparente.

Um **MOM (*Message-Oriented Middleware*)** baseia-se na troca assíncrona de dados por meio de mensagens contidas em filas, diferentemente do modelo síncrono convencional (chamada de procedimento ou requisição HTTP bloqueante), no qual o emissor precisa aguardar o receptor concluir o processamento antes de prosseguir.

Analogia Prática: O Pronto-Socorro e a Sala de Triagem

Imagine um pronto-socorro onde cada ambulância que chega com um paciente crítico precisasse chamar o médico na porta da sala de cirurgia e aguardar o médico terminar a cirurgia para só então entregar a maca. Se o médico estiver ocupado, a ambulância fica presa no estacionamento, impedida de socorrer outros pacientes na cidade.
O **MOM** atua exatamente como a **sala de triagem com protocolo seguro**: a ambulância (Produtor) entrega a ficha clínica na recepção da triagem (RabbitMQ) e é liberada em fração de segundos. A recepção organiza as fichas em ordem de chegada numa fila segura. Assim que um médico (Consumidor) conclui um procedimento, ele retira a próxima ficha disponível e realiza o atendimento. Nenhuma ficha se perde e nenhuma ambulância fica retida na porta do hospital.

2.2 Protocolo AMQP 0-9-1 e Componentes Implementados

O middleware adota o padrão aberto **AMQP 0-9-1 (*Advanced Message Queuing Protocol*)**, que estabelece as regras formais de serialização, confirmação e roteamento de mensagens. No projeto, a arquitetura foi dividida no módulo `middleware/`:

- **Produtor AMQP (`middleware/produtor.py`):** Lê registros contínuos do conjunto de dados clínico (*dataset* CSV) e envia cada leitura como uma mensagem JSON à central de roteamento (*Exchange*) com a chave de roteamento `dados.paciente`.
- **Broker RabbitMQ (Servidor de Mensageria):** Servidor intermediário encarregado de receber as mensagens, aplicar as regras de roteamento e armazená-las de forma persistente na fila `fila_dados_pacientes`.
- **Consumidor AMQP (`middleware/consumidor.py`):** Processo de escuta contínua que drena os registros da fila e os encaminha para processamento posterior no balanceador de carga.

Analogia Prática: Exchange, Routing Key e Fila como Central dos Correios

Para compreender a dinâmica interna do RabbitMQ:

- **Exchange (Central de Distribuição):** É a esteira rolante dos correios que recebe todas as cartas que chegam à agência. Ela não guarda as cartas; apenas olha para a etiqueta e decide para onde despachar.
- **Routing Key (CEP ou Etiqueta de Destino):** O código impresso no envelope (no projeto, `dados.paciente`) indicando o assunto ou destino da correspondência.
- **Queue / Fila (Caixa Postal ou Escaninho):** A gaveta física onde as cartas ficam guardadas de forma segura e ordenada até que o destinatário venha retirá-las.

2.3 Políticas de Confiabilidade e Tolerância a Falhas

Para garantir que nenhum dado clínico seja extraviado por instabilidade de rede ou pane de energia, foram aplicadas quatro diretrizes estritas:

1. **Persistência de Mensagens (`delivery_mode=2`):** Cada mensagem recebida pelo broker é gravada imediatamente em disco rígido (armazenamento não-volátil), garantindo que ela sobreviva mesmo que o RabbitMQ seja reiniciado abruptamente.

   Analogia Prática: Livro Ata vs. Bilhete Adesivo

   Em vez de colar um bilhete de papel adesivo na parede que pode cair com uma rajada de vento ou apagar-se no escuro (mensagem apenas em memória RAM), cada prontuário é transcrito com tinta indelével em um livro de registro em cofre de aço (persistência em disco).

2. **Confirmação Explícita de Recebimento (*Manual Acknowledgement* — `basic_ack`):** O consumidor não descarta a mensagem da fila ao lê-la. O RabbitMQ mantém a mensagem sob custódia e só a remove em definitivo após receber uma confirmação formal (`basic_ack`) indicando que o servidor backend processou e salvou os dados com sucesso (código HTTP 200).

3. **Rejeição com Reenfileiramento (*Negative Acknowledgement* — `basic_nack`):** Caso ocorra timeout, queda de conexão ou indisponibilidade temporária de backend, o consumidor emite a instrução de rejeição com a flag de retenção (`requeue=True`). O broker reinsere a mensagem no topo da fila para nova tentativa imediata por outro nó disponível.

Analogia Prática: Assinatura do Recibo de Entrega

É o equivalente a assinar o canhoto do Sedex apenas após abrir a embalagem e conferir que o frasco do medicamento chegou intacto. Se a caixa estiver avariada, o recepcionista recusa a assinatura (*NACK*) e a encomenda retorna para o almoxarifado para reenvio sem prejuízo.

4. **Controle de Fluxo e Prevenção de Sobrecarga (`prefetch_count=1`):** Instrução que limita o broker a entregar apenas uma mensagem por vez a cada consumidor ativo. Novas mensagens só são despachadas após o processamento da mensagem corrente.

Analogia Prática: Uma Pasta Médica por Vez na Mesa

Impede que um único profissional retire 50 prontuários de uma vez da triagem e acumule uma pilha sobrecarregada em sua mesa enquanto outros colegas de plantão permanecem ociosos. Cada um atende um paciente por vez, promovendo divisão justa e evitando esgotamento de memória (*backpressure*).

3 Virtualização e Topologia Segregada em 6 Máquinas Virtuais

3.1 Do Compartilhamento de Recursos ao Isolamento Real de Falhas

Na primeira versão da infraestrutura, os três servidores de aplicação operavam como processos distintos dentro de uma única Máquina Virtual denominada `vm-servidores`. Sob os preceitos de Sistemas Distribuídos, esse modelo configurava um falso isolamento: embora fossem portas lógicas distintas, qualquer exaustão de memória RAM, saturação de CPU ou pane no kernel da máquina virtual derrubaria todos os três servidores simultaneamente.

A reformulação implementada no arquivo `Vagrantfile` segregou o sistema em **6 Máquinas Virtuais totalmente autônomas**, cada qual executando seu próprio sistema operacional Ubuntu, pilha TCP/IP e alocação dedicada de memória e processamento.

Analogia Prática: Prédio de Apartamentos vs. Casas Independentes

Antes, os três servidores eram como três pessoas dividindo o mesmo quarto de uma casa: se o cano daquele cômodo estourasse ou houvesse um curto-circuito na fiação (pane do SO), todos os moradores ficavam desamparados.
Com as 6 VMs, criamos um condomínio fechado com 6 casas muradas independentes. Se a casa de um servidor sofrer uma pane isolada, as outras 5 casas continuam com energia elétrica, abastecimento e portas abertas para atendimento normal.

3.2 Topologia de Rede Privada e Tabela de Endereçamento

Todas as máquinas virtuais comunicam-se através de uma rede interna privada (*Host-Only Network* no segmento `192.168.56.0/24`). Esta configuração garante isolamento contra ruídos externos e simula fielmente a rede local cabeada de um complexo hospitalar.

Máquina Virtual	Endereço IP	Porta(s)	Papel Arquitetural e Serviços Alocados
`vm-cliente`	`192.168.56.10`	—	Produtor de telemetria médica (`produtor.py`)
`vm-middleware`	`192.168.56.11`	5672, 15672	Broker RabbitMQ (Protocolo AMQP e Painel de Gestão)
`vm-balanceador`	`192.168.56.12`	8000	Consumidor AMQP e Load Balancer (Round Robin)
`vm-servidor1`	`192.168.56.13`	8001	Servidor Backend 01 (FastAPI) e Banco `server1.db`
`vm-servidor2`	`192.168.56.14`	8002	Servidor Backend 02 (FastAPI) e Banco `server2.db`
`vm-servidor3`	`192.168.56.15`	8003	Servidor Backend 03 (FastAPI) e Banco `server3.db`

Tabela 1: Mapeamento topológico das 6 Máquinas Virtuais independentes do projeto.

3.3 Distinção entre Infraestrutura como Código (IaC) e Execução Local

Para fins de avaliação do projeto, é fundamental compreender a coexistência harmoniosa de dois ambientes complementares:

- **Ambiente Formal de Infraestrutura como Código (`Vagrantfile`):** A especificação das 6 máquinas virtuais com provisionamento automatizado é integralmente funcional. Ao invocar o comando `vagrant up`, o hipervisor VirtualBox inicializa os 6 nós virtuais com adaptadores de rede físicos e serviços independentes.
- **Ambiente de Simulação Ágil (`executar.sh`):** Para permitir validações rápidas de lógica sem exigir 6 GB de memória RAM dedicada e virtualização aninhada na estação de trabalho do desenvolvedor, o script `executar.sh` orquestra os mesmos componentes como processos isolados no sistema operacional do host, vinculados às portas de rede locais 8000 a 8003 e comunicando-se com a instância local do RabbitMQ.

4 Camada de Apresentação e Visualização em Tempo Real

4.1 Observabilidade em Sistemas Distribuídos e Protocolo SSE

Em sistemas distribuídos, a observabilidade consiste na capacidade de inferir o estado interno de um nó a partir das suas saídas externas. Para viabilizar a visualização didática do fluxo de telemetria médica, a interface gráfica (`visualizacao/index.html`) utiliza **Server-Sent Events (SSE)**, um protocolo baseado em HTTP que mantém uma conexão unidirecional persistente aberta entre o servidor e o navegador.

Analogia Prática: O Monitor Cardíaco Contínuo vs. O Questionamento Frequente

Se o navegador precisasse fazer consultas periódicas de segundo em segundo para saber se há novos dados (*Polling*), seria o mesmo que uma enfermeira entrar a cada dez segundos no quarto do paciente gritando "qual é a pressão arterial agora?". Isso sobrecarrega a rede e desperdiça tempo.
Com o protocolo **SSE**, o navegador comporta-se como um **monitor cardíaco moderno com cabo conectado**: a linha de transmissão permanece aberta e, no milissegundo exato em que um batimento cardíaco ou anomalia clínica é computada, o sinal é enviado instantaneamente para a tela do médico.

4.2 Reformulação da Topologia Gráfica Vetorial (SVG)

O painel vetorial foi integralmente adaptado para refletir a nova realidade de 6 máquinas:

- **Fronteiras Virtuais Explícitas:** Cada uma das 6 VMs é demarcada por um retângulo tracejado estilizado, contendo seu respectivo nome de host (`vm-cliente`, `vm-middleware`, etc.) e endereço IP estático dedicado;
- **Destaque do Middleware:** A exchange, a fila durável e a porta AMQP 5672 ganharam representação visual proeminente, evidenciando o acúmulo temporário e o dreno das mensagens;
- **Animação Física de Pacotes (*Flying Packets*):** Círculos luminosos percorrem o diagrama simulando os pacotes físicos de dados transitando em tempo real: Produtor → Broker RabbitMQ → Balanceador → Servidor Backend selecionado.

5 Análise Crítica e Conceitual de Escalabilidade

5.1 Escalabilidade Horizontal versus Vertical

A capacidade de um sistema em lidar com o aumento de demanda define sua escalabilidade:

- **Escalabilidade Vertical (*Scale-Up*):** Adicionar mais recursos a uma única máquina (mais processadores, mais memória RAM). Possui barreiras físicas estritas e custo que cresce exponencialmente.

- **Escalabilidade Horizontal (*Scale-Out*):** Adicionar mais máquinas trabalhando em conjunto. É a base dos sistemas distribuídos modernos e a arquitetura adotada em nosso projeto.

Analogia Prática: Trocar o Motor do Veículo vs. Ampliar a Frota

A escalabilidade vertical é equivalente a trocar o motor de uma ambulância comum pelo motor de um jato supersônico. Além de ter um limite físico intransponível, o custo financeiro é exorbitante e, se aquele veículo único pifar, o hospital inteiro fica sem transporte.
A escalabilidade horizontal equivale a adquirir uma frota com 5 novas ambulâncias de fábrica. A capacidade de atendimento multiplica-se por cinco e, caso uma ambulância precise de manutenção, as outras quatro continuam operando sem qualquer interrupção.

5.2 A Escalabilidade do Projeto é Automática?

Não. A escalabilidade do sistema é horizontal manual (configurável), e não automática (*Auto-Scaling*).

Analogia Prática: O Gerente da Agência Bancária vs. A Catraca Inteligente

Pense em uma agência bancária operando com 3 guichês de atendimento. Se a fila de espera começar a dobrar a esquina, o gerente da agência pode autorizar a abertura de um 4º guichê para diluir o tempo de espera.
Isso é **escalabilidade horizontal manual**: o sistema possui a flexibilidade estrutural para comportar novos caixas sem precisar quebrar as regras de atendimento do banco, mas a abertura depende de uma decisão deliberada do gerente.
A **escalabilidade automática (*Auto-Scaling*)**, por outro lado, seria o equivalente a uma catraca equipada com inteligência computacional que cronometrasse o tamanho da fila e, autonomamente, convocasse novos atendentes e abrisse novos guichês sem qualquer toque humano.

5.3 Capacidades Atuais e Requisitos para Auto-Scaling Pleno

Na implementação atual do projeto:

- **Expansão de Servidores Backend:** Pode-se iniciar um Servidor 04 (porta 8004) e registrá-lo na inicialização do balanceador (`-servidores`). O balanceador redistribuirá instantaneamente a carga (passando de 33,3% para 25% por servidor), sem necessidade de reescrever ou recompilar o código;
- **Elasticidade de Consumidores:** Múltiplos processos consumidores (`consumidor.py`) podem se conectar concorrentemente à mesma fila do RabbitMQ (*Competing Consumers Pattern*), e o broker dividirá a entrega das mensagens dinamicamente entre eles.

Para atingir a **escalabilidade automática plena**, o sistema exigiria três componentes de infraestrutura de nível de produção que ultrapassam o escopo pedagógico deste marco:

1. **Sensor de Métricas e Gatilhos (*Triggers*):** Um agente monitor que avaliasse a profundidade da fila do RabbitMQ e o tempo de resposta das requisições para disparar ordens de escala;
2. **Orquestrador Dinâmico de Contêineres:** Uma ferramenta de orquestração (como Kubernetes HPA ou Docker Swarm) encarregada de clonar e iniciar novos nós automaticamente sob demanda;
3. **Descoberta Dinâmica de Serviços (*Service Discovery*):** Um registro centralizado em tempo real (como HashiCorp Consul ou Apache Zookeeper) que notificasse o balanceador da chegada do novo servidor, atualizando o algoritmo Round Robin sem intervenção humana ou reinicialização de processos.

6 Conclusão e Síntese Comparativa

As modificações consolidadas elevaram o **Sistema Distribuído para Auxílio de Dados Hospitalares** de um modelo de protótipo síncrono inicial para uma arquitetura distribuída resiliente, desacoplada e aderente às melhores práticas de Engenharia de Sistemas Distribuídos.

A Tabela 2 sintetiza as transformações técnicas entre a versão original e o estado atual do projeto:

Dimensão Técnica	**Estado Inicial (Protótipo)**	**Estado Atual (Evolução com MOM e 6 VMs)**
Comunicação	Síncrona via HTTP (Acoplada e bloqueante)	Assíncrona via protocolo AMQP 0-9-1 com desacoplamento temporal pleno
Bufferização de Tráfego	Inexistente (picos de telemetria sobrecarregavam portas)	Fila durável RabbitMQ com persistência em disco (`delivery_mode=2`)
Garantia de Entrega	Frágil (quedas de servidor causavam perda de leituras)	Confirmação explícita (`basic_ack`) e reenfileiramento (`basic_nack`)
Controle de Fluxo	Sem limite (risco de saturação de memória)	`prefetch_count=1` garantindo entrega justa e prevenção de sobrecarga
Ambiente Virtual	Servidores concorrendo na mesma VM	6 Máquinas Virtuais totalmente isoladas com sistemas operacionais próprios
Segmentação de Rede	Portas lógicas compartilhadas	Rede privada cabeada dedicada (*Host-Only* `192.168.56.0/24`)
Observabilidade	Visualização genérica de blocos	Painel SVG interativo com fronteiras reais de 6 VMs e conexão persistente SSE
Modelo de Escala	Fixo e acoplado	Horizontal manual configurável com divisão proporcional de carga

Tabela 2: Síntese comparativa da evolução arquitetural do sistema distribuído.

Em síntese, a integração do RabbitMQ como middleware de mensageria eliminou os riscos de perda silenciosa de sinais vitais críticos em momentos de sobrecarga. Concomitantemente, o desmembramento da infraestrutura em 6 Máquinas Virtuais independentes no `Vagrantfile` concedeu tolerância e isolamento real de falhas físicas e de sistema operacional, consolidando uma plataforma hospitalar robusta, auditável e didaticamente exemplar.
